

根号算法选讲

湖南师大附中 高铭鸿

2026.4.27

根号算法的核心思想

- 平衡：时间与空间、预处理与查询、大与小之间的平衡

根号算法的核心思想

- 平衡：时间与空间、预处理与查询、大与小之间的平衡
- 常见技巧：分块、莫队、根号分类、整除分块、定期重构

根号算法的核心思想

- 平衡：时间与空间、预处理与查询、大与小之间的平衡
- 常见技巧：分块、莫队、根号分类、整除分块、定期重构
- 复杂度通常为 $O(n\sqrt{n})$ 或 $O(\sqrt{n})$ 每操作

分块算法简介

- 将序列分成 $\lceil n/S \rceil$ 块，每块大小 $S \approx \sqrt{n}$

分块算法简介

- 将序列分成 $\lceil n/S \rceil$ 块，每块大小 $S \approx \sqrt{n}$
- 整块整体处理，散块暴力处理——“大段维护，小段朴素”

分块算法简介

- 将序列分成 $\lceil n/S \rceil$ 块，每块大小 $S \approx \sqrt{n}$
- 整块整体处理，散块暴力处理——“大段维护，小段朴素”
- 比线段树更灵活，可维护不可合并的信息

分块实现细节

- 块大小通常取 $S = \lfloor \sqrt{n} \rfloor$

分块实现细节

- 块大小通常取 $S = \lfloor \sqrt{n} \rfloor$
- 维护: $bel[i]$ (所属块), $L[b], R[b]$ (块边界), $tag[b]$ (懒标记)

分块实现细节

- 块大小通常取 $S = \lfloor \sqrt{n} \rfloor$
- 维护: $bel[i]$ (所属块), $L[b], R[b]$ (块边界), $tag[b]$ (懒标记)
- 修改/查询: 整块用懒标记, 散块暴力

分块实现细节

- 块大小通常取 $S = \lfloor \sqrt{n} \rfloor$
- 维护: $bel[i]$ (所属块), $L[b], R[b]$ (块边界), $tag[b]$ (懒标记)
- 修改/查询: 整块用懒标记, 散块暴力
- 复杂度: $O(n\sqrt{n})$

分块练习题

- LOJ#6280 数列分块入门 4：区间加法，区间求和。

分块练习题

- **LOJ#6280 数列分块入门 4**: 区间加法, 区间求和。

题解: 维护块内和, 整块打标记并更新和, 散块暴力加并更新块和。

分块练习题

- **LOJ#6280 数列分块入门 4**: 区间加法, 区间求和。
题解: 维护块内和, 整块打标记并更新和, 散块暴力加并更新块和。
- **LOJ#6278 数列分块入门 2**: 区间加法, 查询区间内小于某值的元素个数。

分块练习题

- **LOJ#6280 数列分块入门 4**: 区间加法, 区间求和。

题解: 维护块内和, 整块打标记并更新和, 散块暴力加并更新块和。

- **LOJ#6278 数列分块入门 2**: 区间加法, 查询区间内小于某值的元素个数。

题解: 每块内维护排序后的数组, 整块二分查找, 散块暴力统计。复杂度 $O((N+Q)\sqrt{N}\log N)$ 。

如果元素值 $\leq \sqrt{N}$, 可以每块维护 Cnt 数组以及前缀和, 复杂度 $O((N+Q)\sqrt{N})$ 。

分块应用题 (Comet OJ Round 13 F)

给定长为 n 的序列 $a_1, a_2, \dots, a_n$, q 次操作:

- ❶ 1 l r: 将 $[l, r]$ 循环左移一位, 即 $a_i \leftarrow a_{i+1}$ ($l \leq i < r$),
 $a_r \leftarrow a_l$ 。
- ❷ 2 l r: 将 $[l, r]$ 中每个元素 $+1$ 。
- ❸ 3 x: 求离 a_x 最近且值与 a_x 相等的另一个元素到 x 的距离
(无则输出 -1)。

$1 \leq n, q \leq 10^5$, 数值范围 $O(n)$ 。

分块应用题 (Comet OJ Round 13 F) 解法

- 将序列分成 $\sqrt{n}$ 块, 每块大小 $B \approx \sqrt{n}$ 。

分块应用题 (Comet OJ Round 13 F) 解法

- 将序列分成 $\sqrt{n}$ 块，每块大小 $B \approx \sqrt{n}$ 。
- 每块维护：
 - 整体加法标记 add ;
 - 左右端点 L, R ;
 - 数组 $\text{cnt}[v]$ 记录块内当前值为 v 的元素个数。

分块应用题 (Comet OJ Round 13 F) 解法

- 将序列分成 $\sqrt{n}$ 块，每块大小 $B \approx \sqrt{n}$ 。
- 每块维护：▪ 整体加法标记 add ；▪ 左右端点 L, R ；▪ 数组 $\text{cnt}[v]$ 记录块内当前值为 v 的元素个数。
- 操作 2 (区间加)：整块 $\text{add} += 1$ ；散块暴力修改并更新对应块的 cnt 数组。

分块应用题 (Comet OJ Round 13 F) 解法

- 将序列分成 $\sqrt{n}$ 块，每块大小 $B \approx \sqrt{n}$ 。
- 每块维护：▪ 整体加法标记 add ；▪ 左右端点 L, R ；▪ 数组 $\text{cnt}[v]$ 记录块内当前值为 v 的元素个数。
- 操作 2 (区间加)：整块 $\text{add} += 1$ ；散块暴力修改并更新对应块的 cnt 数组。
- 操作 1 (区间左移)：整块 $L- = 1, R- = 1$ ；散块暴力模拟并更新 cnt 。如果出现大小超过 $2B$ 的块则打散成两个。

分块应用题 (Comet OJ Round 13 F) 解法

- 将序列分成 $\sqrt{n}$ 块，每块大小 $B \approx \sqrt{n}$ 。
- 每块维护：
 - 整体加法标记 add ;
 - 左右端点 L, R ;
 - 数组 $\text{cnt}[v]$ 记录块内当前值为 v 的元素个数。
- 操作 2 (区间加): 整块 $\text{add} += 1$; 散块暴力修改并更新对应块的 cnt 数组。
- 操作 1 (区间左移): 整块 $L- = 1, R- = 1$; 散块暴力模拟并更新 cnt 。如果出现大小超过 $2B$ 的块则打散成两个。
- 操作 3 (查询): 设 $v = a_x + \text{add}_{\text{bel}[x]}$ 。两个方向从近到远遍历每个块，令 $t = v - \text{add}_{\text{block}}$ ，若该块 $\text{cnt}[t] > 0$ ，则在块内暴力扫描找到最近位置，更新答案，然后 break 。

分块应用题 (Comet OJ Round 13 F) 解法

- 将序列分成 $\sqrt{n}$ 块，每块大小 $B \approx \sqrt{n}$ 。
- 每块维护：
 - 整体加法标记 add ;
 - 左右端点 L, R ;
 - 数组 $\text{cnt}[v]$ 记录块内当前值为 v 的元素个数。
- 操作 2 (区间加): 整块 $\text{add} += 1$; 散块暴力修改并更新对应块的 cnt 数组。
- 操作 1 (区间左移): 整块 $L- = 1, R- = 1$; 散块暴力模拟并更新 cnt 。如果出现大小超过 $2B$ 的块则打散成两个。
- 操作 3 (查询): 设 $v = a_x + \text{add}_{\text{bel}[x]}$ 。两个方向从近到远遍历每个块，令 $t = v - \text{add}_{\text{block}}$ ，若该块 $\text{cnt}[t] > 0$ ，则在块内暴力扫描找到最近位置，更新答案，然后 break 。
- 复杂度: $O((n + q)\sqrt{n})$ 。

序列莫队

- 题目模型一般来说是给定一个 n 的序列和 q 个关于区间的询问。无修改，并且询问可以离线处理。
- 离线 + 排序 + 双指针移动

序列莫队

- 题目模型一般来说是给定一个 n 的序列和 q 个关于区间的询问。无修改，并且询问可以离线处理。
- 离线 + 排序 + 双指针移动
- 将查询按左端点所在块和右端点，即 $(bel[l], r)$ 二元组排序

序列莫队

- 题目模型一般来说是给定一个 n 的序列和 q 个关于区间的询问。无修改，并且询问可以离线处理。
- 离线 + 排序 + 双指针移动
- 将查询按左端点所在块和右端点，即 $(bel[l], r)$ 二元组排序
- 维护一个答案区间 $[L, R]$ ，处理询问 (l, r) 就令 $L \rightarrow l, R \rightarrow r$ ，每次移动指针 $O(1)$ / $O(\log n)$ 维护答案

序列莫队

- 题目模型一般来说是给定一个 n 的序列和 q 个关于区间的询问。无修改，并且询问可以离线处理。
- 离线 + 排序 + 双指针移动
- 将查询按左端点所在块和右端点，即 $(bel[l], r)$ 二元组排序
- 维护一个答案区间 $[L, R]$ ，处理询问 (l, r) 就令 $L \rightarrow l, R \rightarrow r$ ，每次移动指针 $O(1)$ / $O(\log n)$ 维护答案
- 同一个左端点块内，左端点每次移动次数 $O(\sqrt{n})$ ，右端点总移动次数 $O(n)$ ；有 $O(\sqrt{n})$ 次左端点跨块，每次移动次数 $O(n)$ ；所以总移动次数为 $O((n + m)\sqrt{n})$

序列莫队

- 题目模型一般来说是给定一个 n 的序列和 q 个关于区间的询问。无修改，并且询问可以离线处理。
- 离线 + 排序 + 双指针移动
- 将查询按左端点所在块和右端点，即 $(bel[l], r)$ 二元组排序
- 维护一个答案区间 $[L, R]$ ，处理询问 (l, r) 就令 $L \rightarrow l, R \rightarrow r$ ，每次移动指针 $O(1)$ / $O(\log n)$ 维护答案
- 同一个左端点块内，左端点每次移动次数 $O(\sqrt{n})$ ，右端点总移动次数 $O(n)$ ；有 $O(\sqrt{n})$ 次左端点跨块，每次移动次数 $O(n)$ ；所以总移动次数为 $O((n + m)\sqrt{n})$
- 复杂度 $O((n + m)\sqrt{n})$ / $O((n + m)\sqrt{n} \log n)$

扩展莫队

- 带修莫队：增加时间维度，排序关键字为 (左块, 右块, 时间)

扩展莫队

- 带修莫队：增加时间维度，排序关键字为 (左块, 右块, 时间)
- 树上莫队：利用欧拉序（括号序）将树上路径转化为区间

扩展莫队

- 带修莫队：增加时间维度，排序关键字为 (左块, 右块, 时间)
- 树上莫队：利用欧拉序（括号序）将树上路径转化为区间
- 回滚莫队：只支持添加，避免删除操作

莫队练习题

- **Luogu P1494 小 Z 的袜子**: 区间内随机抽两只袜子, 求颜色相同的概率。

莫队练习题

- **Luogu P1494 小 Z 的袜子**：区间内随机抽两只袜子，求颜色相同的概率。

题解： 莫队算法 + 组合数公式。

莫队练习题

- **Luogu P1494 小 Z 的袜子**：区间内随机抽两只袜子，求颜色相同的概率。

题解：莫队算法 + 组合数公式。

设当前区间内颜色 i 出现次数为 $cnt[i]$ ，分母为

$len \cdot (len - 1)$ ，分子为 $\sum cnt[i] \cdot (cnt[i] - 1)$ ，答案即为 $\frac{\sum cnt[i] \cdot (cnt[i] - 1)}{len \cdot (len - 1)}$ 。

莫队练习题

- **Luogu P1494 小 Z 的袜子**: 区间内随机抽两只袜子, 求颜色相同的概率。

题解: 莫队算法 + 组合数公式。

设当前区间内颜色 i 出现次数为 $cnt[i]$, 分母为

$len \cdot (len - 1)$, 分子为 $\sum cnt[i] \cdot (cnt[i] - 1)$, 答案即为 $\frac{\sum cnt[i] \cdot (cnt[i] - 1)}{len \cdot (len - 1)}$ 。

区间转移时, 添加颜色 x 时, 分子增加 $2 \times cnt[x]$; 删除颜色 x 时, 分子减少 $2 \times (cnt[x] - 1)$ 。

莫队练习题 - 带修莫队

- **Luogu P1903 数颜色**: 带修改的区间不同元素个数。

莫队练习题 - 带修莫队

- **Luogu P1903 数颜色**：带修改的区间不同元素个数。

题解：带修莫队，增加时间维度，操作以 $(bel[l], bel[r], id)$ 排序，维护 (L, R, T) 和一个记录元素出现次数的 cnt 数组。

莫队练习题 - 带修莫队

- **Luogu P1903 数颜色**：带修改的区间不同元素个数。

题解：带修莫队，增加时间维度，操作以 $(bel[l], bel[r], id)$ 排序，维护 (L, R, T) 和一个记录元素出现次数的 cnt 数组。当 T 扫描到修改操作时交换当前修改操作的颜色（ T 从小到大则正向修改，从大到小则逆向修改）并维护 cnt 。

莫队练习题 - 带修莫队

- **Luogu P1903 数颜色**：带修改的区间不同元素个数。

题解：带修莫队，增加时间维度，操作以 $(bel[l], bel[r], id)$ 排序，维护 (L, R, T) 和一个记录元素出现次数的 cnt 数组。当 T 扫描到修改操作时交换当前修改操作的颜色（ T 从小到大则正向修改，从大到小则逆向修改）并维护 cnt 。块大小可以设成 $O(n^{\frac{2}{3}})$ ，总复杂度是 $O(n^{\frac{5}{3}})$ 的。同学们可以试着自己推算一下。

莫队练习题 - 树上莫队

- **SP10707 COT2:** 树上路径不同点权。

莫队练习题 - 树上莫队

- **SP10707 COT2:** 树上路径不同点权。

推荐用 $2n - 1$ 版欧拉路径序来将树上的链转化为区间，下面简单介绍一下欧拉路径序。

欧拉路径序 (Euler Tour Sequence, $2n - 1$ 版)

- 定义：DFS 时，每次进入一个节点时记录，每次从子节点返回父节点时也记录父节点（不记录子节点的离开）。

欧拉路径序 (Euler Tour Sequence, $2n - 1$ 版)

- 定义：DFS 时，每次进入一个节点时记录，每次从子节点返回父节点时也记录父节点（不记录子节点的离开）。
- 长度： $2n - 1$ ，每个节点出现次数等于其度数，根节点多一次。

欧拉路径序 (Euler Tour Sequence, $2n - 1$ 版)

- 定义：DFS 时，每次进入一个节点时记录，每次从子节点返回父节点时也记录父节点（不记录子节点的离开）。
- 长度： $2n - 1$ ，每个节点出现次数等于其度数，根节点多一次。
- 示例：树 1-2,1-3,2-4，DFS 顺序（记录点）：
1（进），2（进），4（进），2（返），1（返），3（进），1（返）
序列：1, 2, 4, 2, 1, 3, 1

欧拉路径序 (Euler Tour Sequence, $2n - 1$ 版)

- 与常见欧拉序对比:

类型	记录方式	长度
标准欧拉序	进 + 出	$2n$
仅进入序	只记进	n
路径序	进 + 回溯父节点	$2n - 1$

欧拉路径序 (Euler Tour Sequence, $2n - 1$ 版)

- **核心性质：**序列中相邻的节点在树上也相邻，所以序列中的任意连续子段可以对应树上的一条可经过重复点的路径。

欧拉路径序 (Euler Tour Sequence, $2n - 1$ 版)

- **核心性质：**序列中相邻的节点在树上也相邻，所以序列中的任意连续子段可以对应树上的一条可经过重复点的路径。
- **扩展应用：**LCA 转 RMQ。

欧拉路径序 (Euler Tour Sequence, $2n - 1$ 版)

- **核心性质：**序列中相邻的节点在树上也相邻，所以序列中的任意连续子段可以对应树上的一条可经过重复点的路径。
- **扩展应用：**LCA 转 RMQ。
 - 对每个点记录第一次出现位置 $\text{pos}[u]$ 。
 - 构建深度数组 $\text{dep}[i]$ ，则 $\text{LCA}(u, v)$ 为区间 $[\min(\text{pos}[u], \text{pos}[v]), \max(\text{pos}[u], \text{pos}[v])]$ 中深度最小的节点。
 - 用 ST 表 $O(1)$ 查询 LCA。

莫队练习题 - 树上莫队

- **SPOJ COT2:** 树上路径不同颜色数。

莫队练习题 - 树上莫队

- **SPOJ COT2:** 树上路径不同颜色数。

题解：先求出欧拉路径序 $A_1, A_2, \dots, A_{2n-1}$ ，记 $\text{pos}[u]$ 为第一次访问，路径 (u, v) 对应区间 $[\min(\text{pos}[u], \text{pos}[v]), \max(\text{pos}[u], \text{pos}[v])]$ 。

莫队练习题 - 树上莫队

- **SPOJ COT2:** 树上路径不同颜色数。

题解：先求出欧拉路径序 $A_1, A_2, \dots, A_{2n-1}$ ，记 $\text{pos}[u]$ 为第一次访问，路径 (u, v) 对应区间

$[\min(\text{pos}[u], \text{pos}[v]), \max(\text{pos}[u], \text{pos}[v])]$ 。

一个需要注意的点，欧拉路径序中的区间 $[L, R]$ 扩张 ($L \rightarrow L-1$ 或者 $R \rightarrow R+1$) 时，对应到树上的序列不一定是扩张的，得看新扩张的点 A_{L-1} 或者 A_{R+1} 在不在当前路径上，如果在则路径是从 $A_L \rightarrow A_R$ 扩张到 $A_{L-1} \rightarrow A_R$ 或者 $A_L \rightarrow A_{R+1}$ ，如果不在则路径是收缩的。区间 $[L, R]$ 收缩时的判断同理。

莫队练习题 - 回滚莫队

- **Luogu P5906 [模板] 回滚莫队**：区间内相同的数的最远间隔距离。

莫队练习题 - 回滚莫队

- **Luogu P5906 [模板] 回滚莫队**：区间内相同的数的最远间隔距离。
题解：注意到我们不能像记 cnt 数组一样很简单地处理删除数字，我们只能做到撤销上一次操作。

莫队练习题 - 回滚莫队

- **Luogu P5906 [模板] 回滚莫队**：区间内相同的数的最远间隔距离。

题解：注意到我们不能像记 cnt 数组一样很简单地处理删除数字，我们只能做到撤销上一次操作。

记当前左端点块的询问有 $[l_1, r_1], [l_2, r_2], \dots, [l_k, r_k]$ ，首先有 $r_1 \leq r_2 \leq \dots \leq r_k$ ，然后我们求出 $L = \max\{l_i\}$ 。

莫队练习题 - 回滚莫队

- **Luogu P5906 [模板] 回滚莫队**：区间内相同的数的最远间隔距离。

题解：注意到我们不能像记 cnt 数组一样很简单地处理删除数字，我们只能做到撤销上一次操作。

记当前左端点块的询问有 $[l_1, r_1], [l_2, r_2], \dots, [l_k, r_k]$ ，首先有 $r_1 \leq r_2 \leq \dots \leq r_k$ ，然后我们求出 $L = \max\{l_i\}$ 。

对于 $r_i < L$ 的询问，区间长度是 $O(\sqrt{N})$ 的，暴力处理即可；

莫队练习题 - 回滚莫队

- **Luogu P5906 [模板] 回滚莫队**：区间内相同的数的最远间隔距离。

题解：注意到我们不能像记 cnt 数组一样很简单地处理删除数字，我们只能做到撤销上一次操作。

记当前左端点块的询问有 $[l_1, r_1], [l_2, r_2], \dots, [l_k, r_k]$ ，首先有 $r_1 \leq r_2 \leq \dots \leq r_k$ ，然后我们求出 $L = \max\{l_i\}$ 。

对于 $r_i < L$ 的询问，区间长度是 $O(\sqrt{N})$ 的，暴力处理即可；然后考虑初始化答案区间 $[L, r] = [L, L]$ ，每扫描到一个询问 $[l_i, r_i]$ ，我们首先将答案区间 $[L, r]$ 扩展到 $[L, r_i]$ ，然后因为 $l_i \leq L$ ，我们可以再扩展到 $[l_i, r_i]$ 并计算答案，最后再把答案区间回滚到 $[L, r_i]$ 即可。

根号分治简介

- 设定阈值 $B \approx \sqrt{n}$, 将元素分为“大”和“小”两类

根号分治简介

- 设定阈值 $B \approx \sqrt{n}$, 将元素分为“大”和“小”两类
- “大”的数量少, “小”的规模小

根号分治简介

- 设定阈值 $B \approx \sqrt{n}$ ，将元素分为“大”和“小”两类
- “大”的数量少，“小”的规模小
- 对两类分别采用不同算法，平衡总复杂度

度数分类

- 无向图中，度数 $\geq B$ 的点称为重点，否则为轻点

度数分类

- 无向图中，度数 $\geq B$ 的点称为重点，否则为轻点
- 重点个数 $\leq 2m/B$ ，轻点度数 $\leq B$

度数分类

- 无向图中，度数 $\geq B$ 的点称为重点，否则为轻点
- 重点个数 $\leq 2m/B$ ，轻点度数 $\leq B$
- 查询带修改的邻接信息和时：轻点暴力，重点预处理

度数分类

- 无向图中，度数 $\geq B$ 的点称为重点，否则为轻点
- 重点个数 $\leq 2m/B$ ，轻点度数 $\leq B$
- 查询带修改的邻接信息和时：轻点暴力，重点预处理
- 复杂度 $O((n + m)\sqrt{m})$

串长分类

- 在一个给了很多字符串的题中，总串长为 L ，长度 $> B$ 的串为长串，否则为短串

串长分类

- 在一个给了很多字符串的题中，总串长为 L ，长度 $> B$ 的串为长串，否则为短串
- 长串个数 $\leq L/B$ ，短串长度 $\leq B$

串长分类

- 在一个给了很多字符串的题中，总串长为 L ，长度 $> B$ 的串为长串，否则为短串
- 长串个数 $\leq L/B$ ，短串长度 $\leq B$
- 查询带加串删串的匹配次数和时：维护短串的 Trie 树，询问暴力枚举长度 $\leq B$ 的子串；维护长串的 AC 自动机，无论加删都直接重构，询问就跑一遍 AC 自动机即可

串长分类

- 在一个给了很多字符串的题中，总串长为 L ，长度 $> B$ 的串为长串，否则为短串
- 长串个数 $\leq L/B$ ，短串长度 $\leq B$
- 查询带加串删串的匹配次数和时：维护短串的 Trie 树，询问暴力枚举长度 $\leq B$ 的子串；维护长串的 AC 自动机，无论加删都直接重构，询问就跑一遍 AC 自动机即可
- 复杂度 $O((n + L)\sqrt{L})$

根号分治练习题

- **Luogu P3396** 哈希冲突：维护序列，支持单点修改和查询

$$\sum_{i \bmod p=y} a_i$$

根号分治练习题

- **Luogu P3396 哈希冲突**: 维护序列, 支持单点修改和查询

$$\sum_{i \bmod p=y} a_i$$

题解: 对于修改操作, 维护模数 $p \leq \sqrt{n}$ 时的答案 $Ans_{p,x}$;

对于查询操作, $p \leq \sqrt{n}$ 时 $O(1)$ 查表, $p > \sqrt{n}$ 时暴力求

和。复杂度: $O((n+q)\sqrt{n})$ 。

根号分治练习题

- **Luogu P3396 哈希冲突**: 维护序列, 支持单点修改和查询

$$\sum_{i \bmod p=y} a_i$$

题解: 对于修改操作, 维护模数 $p \leq \sqrt{n}$ 时的答案 $Ans_{p,x}$;

对于查询操作, $p \leq \sqrt{n}$ 时 $O(1)$ 查表, $p > \sqrt{n}$ 时暴力求

和。复杂度: $O((n+q)\sqrt{n})$ 。

- **CF103D Time to Raid Cowavans**: 查询 $\sum_{i=0}^{\lfloor (n-y)/k \rfloor} a_{y+ik}$ 。

根号分治练习题

- **Luogu P3396 哈希冲突：**维护序列，支持单点修改和查询

$$\sum_{i \bmod p=y} a_i。$$

题解：对于修改操作，维护模数 $p \leq \sqrt{n}$ 时的答案 $Ans_{p,x}$ ；
对于查询操作， $p \leq \sqrt{n}$ 时 $O(1)$ 查表， $p > \sqrt{n}$ 时暴力求和。复杂度： $O((n+q)\sqrt{n})$ 。

- **CF103D Time to Raid Cowavans：**查询 $\sum_{i=0}^{\lfloor (n-y)/k \rfloor} a_{y+ik}。$

题解：对 $\leq B$ 的 k 进行后缀和预处理，其中
 $Suf_{k,i} = A_i + Suf_{k,i+k}$ ，这样每次询问就相当于问一个后缀和了。

根号分治练习题

- **Luogu P3396 哈希冲突：**维护序列，支持单点修改和查询

$$\sum_{i \bmod p=y} a_i。$$

题解：对于修改操作，维护模数 $p \leq \sqrt{n}$ 时的答案 $Ans_{p,x}$ ；对于查询操作， $p \leq \sqrt{n}$ 时 $O(1)$ 查表， $p > \sqrt{n}$ 时暴力求和。复杂度： $O((n+q)\sqrt{n})$ 。

- **CF103D Time to Raid Cowavans：**查询 $\sum_{i=0}^{\lfloor (n-y)/k \rfloor} a_{y+ik}。$

题解：对 $\leq B$ 的 k 进行后缀和预处理，其中 $Suf_{k,i} = A_i + Suf_{k,i+k}$ ，这样每次询问就相当于问一个后缀和了。当 $k \leq \sqrt{n}$ 时， $O(1)$ 求后缀和 $Suf_{k,y}$ ， $k > \sqrt{n}$ 时，暴力求和。复杂度： $O((n+q)\sqrt{n})$ 。

根号分治练习题

- **CF103D Time to Raid Cowavans:** 查询 $\sum_{i=0}^{\lfloor (n-y)/k \rfloor} a_{y+ik}$ 。
如果有单点修改怎么办？

根号分治练习题

- **CF103D Time to Raid Cowavans:** 查询 $\sum_{i=0}^{\lfloor (n-y)/k \rfloor} a_{y+ik}$ 。

如果有单点修改怎么办？

题解：对 $\leq B$ 的 k 进行 $\{1, k+1, 2k+1, \dots, 2, k+2, \dots\}$ 的序列重组。这样每次询问就相当于问一个区间和了，同样也考虑分块。

根号分治练习题

- **CF103D Time to Raid Cowavans:** 查询 $\sum_{i=0}^{\lfloor (n-y)/k \rfloor} a_{y+ik}$ 。

如果有单点修改怎么办？

题解：对 $\leq B$ 的 k 进行 $\{1, k+1, 2k+1, \dots, 2, k+2, \dots\}$ 的序列重组。这样每次询问就相当于问一个区间和了，同样也考虑分块。

对于修改操作，枚举每个 $k \leq B$ ， $O(1)$ 修改数组以及维护块内和，复杂度是 $O(B)$ 。

根号分治练习题

- **CF103D Time to Raid Cowavans:** 查询 $\sum_{i=0}^{\lfloor (n-y)/k \rfloor} a_{y+ik}$ 。

如果有单点修改怎么办？

题解：对 $\leq B$ 的 k 进行 $\{1, k+1, 2k+1, \dots, 2, k+2, \dots\}$ 的序列重组。这样每次询问就相当于问一个区间和了，同样也考虑分块。

对于修改操作，枚举每个 $k \leq B$ ， $O(1)$ 修改数组以及维护块内和，复杂度是 $O(B)$ 。对于求和操作，当 $k \leq B$ 时， $O(B + \frac{n}{B})$ 求区间和， $k > B$ 时， $O(\frac{n}{B})$ 暴力求和。

根号分治练习题

- **CF103D Time to Raid Cowavans:** 查询 $\sum_{i=0}^{\lfloor (n-y)/k \rfloor} a_{y+ik}$ 。

如果有单点修改怎么办？

题解：对 $\leq B$ 的 k 进行 $\{1, k+1, 2k+1, \dots, 2, k+2, \dots\}$ 的序列重组。这样每次询问就相当于问一个区间和了，同样也考虑分块。

对于修改操作，枚举每个 $k \leq B$ ， $O(1)$ 修改数组以及维护块内和，复杂度是 $O(B)$ 。对于求和操作，当 $k \leq B$ 时， $O(B + \frac{n}{B})$ 求区间和， $k > B$ 时， $O(\frac{n}{B})$ 暴力求和。

时间复杂度： $O(n\sqrt{n})$ 。

Luogu P16356 「Diligent-OI R3 D」神秘树

问题简述：找一条树上自根向下的路径 $v_0, v_1, \dots, v_k$ ，要求相邻点深度差 $\leq L$ ，最大化 $\sum (a_{v_{i-1}} \bmod a_{v_i})$ 。

Luogu P16356 「Diligent-OI R3 D」神秘树

问题简述：找一条树上自根向下的路径 $v_0, v_1, \dots, v_k$ ，要求相邻点深度差 $\leq L$ ，最大化 $\sum (a_{v_{i-1}} \bmod a_{v_i})$ 。

$$1 \leq a_i, L, n \leq 3 \times 10^5。$$

Luogu P16356 「Diligent-OI R3 D」神秘树

问题简述：找一条树上自根向下的路径 $v_0, v_1, \dots, v_k$ ，要求相邻点深度差 $\leq L$ ，最大化 $\sum (a_{v_{i-1}} \bmod a_{v_i})$ 。

$$1 \leq a_i, L, n \leq 3 \times 10^5。$$

核心 DP: $f_u = \max_{\substack{v \text{ 是 } u \text{ 的祖先} \\ \text{dep}_u - \text{dep}_v \leq L}} (f_v + (a_v \bmod a_u))$, $f_1 = 0$ 。

Luogu P16356 「Diligent-OI R3 D」神秘树

问题简述：找一条树上自根向下的路径 $v_0, v_1, \dots, v_k$ ，要求相邻点深度差 $\leq L$ ，最大化 $\sum (a_{v_{i-1}} \bmod a_{v_i})$ 。

$$1 \leq a_i, L, n \leq 3 \times 10^5。$$

核心 DP： $f_u = \max_{\substack{v \text{ 是 } u \text{ 的祖先} \\ \text{dep}_u - \text{dep}_v \leq L}} (f_v + (a_v \bmod a_u))$ ， $f_1 = 0$ 。

根号分治：记 $V = \max a$ ，设阈值 $S = \lfloor \sqrt{V} \rfloor \approx 300$ ，按 a_u 大小分两类处理转移。

Luogu P16356 「Diligent-OI R3 D」神秘树

问题简述：找一条树上自根向下的路径 $v_0, v_1, \dots, v_k$ ，要求相邻点深度差 $\leq L$ ，最大化 $\sum (a_{v_{i-1}} \bmod a_{v_i})$ 。

$$1 \leq a_i, L, n \leq 3 \times 10^5。$$

核心 DP： $f_u = \max_{\substack{v \text{ 是 } u \text{ 的祖先} \\ \text{dep}_u - \text{dep}_v \leq L}} (f_v + (a_v \bmod a_u))$ ， $f_1 = 0$ 。

根号分治：记 $V = \max a$ ，设阈值 $S = \lfloor \sqrt{V} \rfloor \approx 300$ ，按 a_u 大小分两类处理转移。

题外话：有利用 $x \bmod z \leq x \bmod y + y \bmod z (x \leq 2y)$ 的不等式的 $O(n \log n)$ 做法。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 小模数 ($a_u < S$):

Luogu P16356 「Diligent-OI R3 D」神秘树

- 小模数 ($a_u < S$):
 - 对每个 $i \in [1, S]$ 维护数组 $g_i[\text{dep}] = f_v + (a_v \bmod i)$ 。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 小模数 ($a_u < S$):
 - 对每个 $i \in [1, S]$ 维护数组 $g_i[\text{dep}] = f_v + (a_v \bmod i)$ 。
 - 查询时需要求区间 $g_{a_u}[\text{dep}_u - L, \text{dep}_u - 1]$ 的最大值。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 小模数 ($a_u < S$):
 - 对每个 $i \in [1, S]$ 维护数组 $g_i[\text{dep}] = f_v + (a_v \bmod i)$ 。
 - 查询时需要求区间 $g_{a_u}[\text{dep}_u - L, \text{dep}_u - 1]$ 的最大值。
 - 可以按深度分块，整块取块 $\max$ ，散块暴力。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 小模数 ($a_u < S$):
 - 对每个 $i \in [1, S]$ 维护数组 $g_i[\text{dep}] = f_v + (a_v \bmod i)$ 。
 - 查询时需要求区间 $g_{a_u}[\text{dep}_u - L, \text{dep}_u - 1]$ 的最大值。
 - 可以按深度分块，整块取块 $\max$ ，散块暴力。
 - 复杂度每次 $O(S + \frac{n}{S})$ 。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 大模数 ($a_u \geq S$):
 - 利用 $a_v \bmod a_u = a_v - \lfloor a_v/a_u \rfloor \cdot a_u$ 。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 大模数 ($a_u \geq S$):
 - 利用 $a_v \bmod a_u = a_v - \lfloor a_v/a_u \rfloor \cdot a_u$ 。
 - 枚举 $k = \lfloor a_v/a_u \rfloor$, 则 $a_v \in [ka_u, (k+1)a_u - 1]$ 。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 大模数 ($a_u \geq S$):
 - 利用 $a_v \bmod a_u = a_v - \lfloor a_v/a_u \rfloor \cdot a_u$ 。
 - 枚举 $k = \lfloor a_v/a_u \rfloor$ ，则 $a_v \in [ka_u, (k+1)a_u - 1]$ 。
 - 需要查询权值区间内 $(f_v + a_v)$ 的最大值。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 大模数 ($a_u \geq S$):
 - 利用 $a_v \bmod a_u = a_v - \lfloor a_v/a_u \rfloor \cdot a_u$ 。
 - 枚举 $k = \lfloor a_v/a_u \rfloor$ ，则 $a_v \in [ka_u, (k+1)a_u - 1]$ 。
 - 需要查询权值区间内 $(f_v + a_v)$ 的最大值。
 - 按照权值分块，并维护块内每个元素的前缀和与后缀和。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 大模数 ($a_u \geq S$):
 - 利用 $a_v \bmod a_u = a_v - \lfloor a_v/a_u \rfloor \cdot a_u$ 。
 - 枚举 $k = \lfloor a_v/a_u \rfloor$ ，则 $a_v \in [ka_u, (k+1)a_u - 1]$ 。
 - 需要查询权值区间内 $(f_v + a_v)$ 的最大值。
 - 按照权值分块，并维护块内每个元素的前缀和与后缀和。
 - 枚举次数 $\frac{V}{a_u} \leq \frac{V}{S}$ ，访问的块的个数 $\frac{V}{S}$ 。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 大模数 ($a_u \geq S$):
 - 利用 $a_v \bmod a_u = a_v - \lfloor a_v/a_u \rfloor \cdot a_u$ 。
 - 枚举 $k = \lfloor a_v/a_u \rfloor$ ，则 $a_v \in [ka_u, (k+1)a_u - 1]$ 。
 - 需要查询权值区间内 $(f_v + a_v)$ 的最大值。
 - 按照权值分块，并维护块内每个元素的前缀和与后缀和。
 - 枚举次数 $\frac{V}{a_u} \leq \frac{V}{S}$ ，访问的块的个数 $\frac{V}{S}$ 。
 - 每次加点 x 可以暴力维护 a_x 所在的块，复杂度 $O(S)$ 。

Luogu P16356 「Diligent-OI R3 D」神秘树

- 大模数 ($a_u \geq S$):
 - 利用 $a_v \bmod a_u = a_v - \lfloor a_v/a_u \rfloor \cdot a_u$ 。
 - 枚举 $k = \lfloor a_v/a_u \rfloor$ ，则 $a_v \in [ka_u, (k+1)a_u - 1]$ 。
 - 需要查询权值区间内 $(f_v + a_v)$ 的最大值。
 - 按照权值分块，并维护块内每个元素的前缀和与后缀和。
 - 枚举次数 $\frac{V}{a_u} \leq \frac{V}{S}$ ，访问的块的个数 $\frac{V}{S}$ 。
 - 每次加点 x 可以暴力维护 a_x 所在的块，复杂度 $O(S)$ 。
 - 复杂度每次 $O(S + \frac{V}{S})$ 。

Luogu P16356 「Diligent-OI R3 D」神秘树

- DFS 维护深度栈，当深度超过 L 时，从数据结构中移除深度为 $\text{dep} - L$ 的祖先节点 y 。

Luogu P16356 「Diligent-OI R3 D」神秘树

- DFS 维护深度栈，当深度超过 L 时，从数据结构中移除深度为 $\text{dep} - L$ 的祖先节点 y 。
- 对小模数：只做前缀 $\max$ 查询，无需删除。

Luogu P16356 「Diligent-OI R3 D」神秘树

- DFS 维护深度栈，当深度超过 L 时，从数据结构中移除深度为 $\text{dep} - L$ 的祖先节点 y 。
- 对小模数：只做前缀 $\max$ 查询，无需删除。
- 对大模数：需支持删除，给每个权值维护一个数据结构，维护每个权值的最大 $(f_v + a_v)$ ，并维护块内元素的前缀和/后缀和，以及块最大值。

Luogu P16356 「Diligent-OI R3 D」神秘树

- DFS 维护深度栈，当深度超过 L 时，从数据结构中移除深度为 $\text{dep} - L$ 的祖先节点 y 。
- 对小模数：只做前缀 max 查询，无需删除。
- 对大模数：需支持删除，给每个权值维护一个数据结构，维护每个权值的最大 $(f_v + a_v)$ ，并维护块内元素的前缀和/后缀和，以及块最大值。
- 回溯时将被移除的节点 y 重新插入，以及删除当前节点。

Luogu P16356 「Diligent-OI R3 D」神秘树

- DFS 维护深度栈，当深度超过 L 时，从数据结构中移除深度为 $\text{dep} - L$ 的祖先节点 y 。
- 对小模数：只做前缀 $\max$ 查询，无需删除。
- 对大模数：需支持删除，给每个权值维护一个数据结构，维护每个权值的最大 $(f_v + a_v)$ ，并维护块内元素的前缀和/后缀和，以及块最大值。
- 回溯时将被移除的节点 y 重新插入，以及删除当前节点。
- 一个 trick：可以令小模数的判定为 $\leq S + 50$ ，因为小模数部分的常数小于大模数部分的常数，这样微调一下就可以避免被 a_i 全部都是略大于 S 的大模数的极限数据卡掉。

Luogu P16356 「Diligent-OI R3 D」神秘树

- DFS 维护深度栈，当深度超过 L 时，从数据结构中移除深度为 $\text{dep} - L$ 的祖先节点 y 。
- 对小模数：只做前缀 $\max$ 查询，无需删除。
- 对大模数：需支持删除，给每个权值维护一个数据结构，维护每个权值的最大 $(f_v + a_v)$ ，并维护块内元素的前缀和/后缀和，以及块最大值。
- 回溯时将被移除的节点 y 重新插入，以及删除当前节点。
- 一个 trick：可以令小模数的判定为 $\leq S + 50$ ，因为小模数部分的常数小于大模数部分的常数，这样微调一下就可以避免被 a_i 全部都是略大于 S 的大模数的极限数据卡掉。
- 时空复杂度： $O(V + n\sqrt{n + V})$ 。

三元环枚举方法

- **Luogu P1989** 无向图三元环计数
- 按度数定向：从度数小的点指向大的（相同则按编号）

三元环枚举方法

- **Luogu P1989** 无向图三元环计数
- 按度数定向：从度数小的点指向大的（相同则按编号）
- 枚举每个点 u ，标记其出邻点 v

三元环枚举方法

- **Luogu P1989** 无向图三元环计数
- 按度数定向：从度数小的点指向大的（相同则按编号）
- 枚举每个点 u ，标记其出邻点 v
- 枚举 u 的出邻点 v ，再枚举 v 的出邻点 w ，若 w 被标记则找到一个三元环 $(u - v - w)$

三元环枚举方法

- **Luogu P1989** 无向图三元环计数
- 按度数定向：从度数小的点指向大的（相同则按编号）
- 枚举每个点 u ，标记其出邻点 v
- 枚举 u 的出邻点 v ，再枚举 v 的出邻点 w ，若 w 被标记则找到一个三元环 $(u - v - w)$
- 每个点 u 的出邻点个数是 $\leq \sqrt{m}$ 的（轻点则显然，重点只会出邻重点，个数也是 $O(\sqrt{m})$ 的），每一条边对复杂度的贡献最多为 $O(\sqrt{m})$ ，所以总复杂度为 $O(m\sqrt{m})$

四元环计数方法

- 类似三元环枚举方法，同样按度数定向（度数小指向大）

四元环计数方法

- 类似三元环枚举方法，同样按度数定向（度数小指向大）
- 枚举每个点 u ，遍历其原本相邻的点 v ，再遍历 v 的出邻点 w ，为了去重，这里只保留 $(deg_w, w) > (deg_u, u)$ 的 w ，然后令 $cnt[w]++$

四元环计数方法

- 类似三元环枚举方法，同样按度数定向（度数小指向大）
- 枚举每个点 u ，遍历其原本相邻的点 v ，再遍历 v 的出邻点 w ，为了去重，这里只保留 $(deg_w, w) > (deg_u, u)$ 的 w ，然后令 $cnt[w]++$
- 对于固定的 u ，遍历完所有 v 后，统计所有 w 对答案的贡献： $\sum_w \binom{cnt[w]}{2}$ ，即从 $cnt[w]$ 个 v 中任选两个，它们与 u 和 w 构成四元环

四元环计数方法

- 类似三元环枚举方法，同样按度数定向（度数小指向大）
- 枚举每个点 u ，遍历其原本相邻的点 v ，再遍历 v 的出邻点 w ，为了去重，这里只保留 $(deg_w, w) > (deg_u, u)$ 的 w ，然后令 $cnt[w]++$
- 对于固定的 u ，遍历完所有 v 后，统计所有 w 对答案的贡献： $\sum_w \binom{cnt[w]}{2}$ ，即从 $cnt[w]$ 个 v 中任选两个，它们与 u 和 w 构成四元环
- 每个环只会被 (deg_w, w) 最大的点 w 统计，所以这样做不用额外去重

四元环计数方法

- 类似三元环枚举方法，同样按度数定向（度数小指向大）
- 枚举每个点 u ，遍历其原本相邻的点 v ，再遍历 v 的出邻点 w ，为了去重，这里只保留 $(deg_w, w) > (deg_u, u)$ 的 w ，然后令 $cnt[w]++$
- 对于固定的 u ，遍历完所有 v 后，统计所有 w 对答案的贡献： $\sum_w \binom{cnt[w]}{2}$ ，即从 $cnt[w]$ 个 v 中任选两个，它们与 u 和 w 构成四元环
- 每个环只会被 (deg_w, w) 最大的点 w 统计，所以这样做不用额外去重
- 复杂度 $O(m\sqrt{m})$

区间众数

- **Luogu P5048 [Ynoi2019]** 在线询问区间众数出现次数，值域 $O(n)$ ，空间要求 $O(n)$

区间众数

- **Luogu P5048 [Ynoi2019]** 在线询问区间众数出现次数，值域 $O(n)$ ，空间要求 $O(n)$
- 预处理块间答案： $f[i][j]$ 表示第 $i \sim j$ 块的众数出现次数

区间众数

- **Luogu P5048 [Ynoi2019]** 在线询问区间众数出现次数，值域 $O(n)$ ，空间要求 $O(n)$
- 预处理块间答案： $f[i][j]$ 表示第 $i \sim j$ 块的众数出现次数
- 预处理每个值 x 出现的位置列表 Vec_x ，以及每个数字 A_i 在列表 Vec_{A_i} 中的出现位置 P_i

区间众数

- **Luogu P5048 [Ynoi2019]** 在线询问区间众数出现次数，值域 $O(n)$ ，空间要求 $O(n)$
- 预处理块间答案： $f[i][j]$ 表示第 $i \sim j$ 块的众数出现次数
- 预处理每个值 x 出现的位置列表 Vec_x ，以及每个数字 A_i 在列表 Vec_{A_i} 中的出现位置 P_i
- 查询时：整块部分直接取预处理的答案 ans ；散点部分，考虑其是否会对答案造成更新，对于左边的散点 i ，如果 $P_i + ans < |Vec_{A_i}| \wedge Vec_{A_i}[Pos_i + ans] \leq r$ ，说明 A_i 在 $[l, r]$ 范围内的出现次数至少是 $ans + 1$ ，令 $ans++$ ，然后继续判断，直到更新不了为止，右边的散点类似

区间众数

- 预处理阶段，每个 $f[i][i \sim \sqrt{n}]$ 的计算都是 $O(n)$ 的，总共只会存 $O(\sqrt{n}) \times O(\sqrt{n}) = O(n)$ 个块间答案

区间众数

- 预处理阶段，每个 $f[i][i \sim \sqrt{n}]$ 的计算都是 $O(n)$ 的，总共只会存 $O(\sqrt{n}) \times O(\sqrt{n}) = O(n)$ 个块间答案
- 每个询问的 $ans \leq f[i][j] + cnt$ ，其中 $cnt \leq 2\sqrt{n}$ 是散点个数，所以散点个数和答案的更新次数都是 $O(\sqrt{n})$ 的

区间众数

- 预处理阶段，每个 $f[i][i \sim \sqrt{n}]$ 的计算都是 $O(n)$ 的，总共只会存 $O(\sqrt{n}) \times O(\sqrt{n}) = O(n)$ 个块间答案
- 每个询问的 $ans \leq f[i][j] + cnt$ ，其中 $cnt \leq 2\sqrt{n}$ 是散点个数，所以散点个数和答案的更新次数都是 $O(\sqrt{n})$ 的
- 证明：反证法，如果一个询问的 $ans > f[i][j] + cnt$ ，那么说明在 $i \sim j$ 块之间存在一个出现次数 $\geq ans - cnt > f[i][j]$ 的数字，与 $i \sim j$ 块之间众数出现次数为 $f[i][j]$ 矛盾

区间众数

- 预处理阶段，每个 $f[i][i \sim \sqrt{n}]$ 的计算都是 $O(n)$ 的，总共只会存 $O(\sqrt{n}) \times O(\sqrt{n}) = O(n)$ 个块间答案
- 每个询问的 $ans \leq f[i][j] + cnt$ ，其中 $cnt \leq 2\sqrt{n}$ 是散点个数，所以散点个数和答案的更新次数都是 $O(\sqrt{n})$ 的
- 证明：反证法，如果一个询问的 $ans > f[i][j] + cnt$ ，那么说明在 $i \sim j$ 块之间存在一个出现次数 $\geq ans - cnt > f[i][j]$ 的数字，与 $i \sim j$ 块之间众数出现次数为 $f[i][j]$ 矛盾
- 时间复杂度： $O((n + m)\sqrt{n})$

数论分块 (整除分块)

- 计算 $\sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor$

数论分块 (整除分块)

- 计算 $\sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor$
- 观察: $\lfloor n/i \rfloor$ 只有 $O(\sqrt{n})$ 种不同取值, 且连续

数论分块 (整除分块)

- 计算 $\sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor$
- 观察: $\lfloor n/i \rfloor$ 只有 $O(\sqrt{n})$ 种不同取值, 且连续
- 对每个左端点 l , 右端点 $r = \left\lfloor \frac{n}{\lfloor n/l \rfloor} \right\rfloor$

数论分块 (整除分块)

- 计算 $\sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor$
- 观察: $\lfloor n/i \rfloor$ 只有 $O(\sqrt{n})$ 种不同取值, 且连续
- 对每个左端点 l , 右端点 $r = \left\lfloor \frac{n}{\lfloor n/l \rfloor} \right\rfloor$
- 复杂度 $O(\sqrt{n})$

数论分块（整除分块）

举例： $n = 10$ 的分块过程

数论分块 (整除分块)

举例: $n = 10$ 的分块过程

- $l = 1, r = 1, \frac{n}{i}$ 值 10, 对答案贡献 $10 \times 1 = 10$

数论分块 (整除分块)

举例: $n = 10$ 的分块过程

- $l = 1, r = 1, \frac{n}{i}$ 值 10, 对答案贡献 $10 \times 1 = 10$
- $l = 2, r = 2, \frac{n}{i}$ 值 5, 对答案贡献 $5 \times 1 = 5$

数论分块 (整除分块)

举例: $n = 10$ 的分块过程

- $l = 1, r = 1, \frac{n}{i}$ 值 10, 对答案贡献 $10 \times 1 = 10$
- $l = 2, r = 2, \frac{n}{i}$ 值 5, 对答案贡献 $5 \times 1 = 5$
- $l = 3, r = 3, \frac{n}{i}$ 值 3, 对答案贡献 $3 \times 1 = 3$

数论分块 (整除分块)

举例: $n = 10$ 的分块过程

- $l = 1, r = 1, \frac{n}{i}$ 值 10, 对答案贡献 $10 \times 1 = 10$
- $l = 2, r = 2, \frac{n}{i}$ 值 5, 对答案贡献 $5 \times 1 = 5$
- $l = 3, r = 3, \frac{n}{i}$ 值 3, 对答案贡献 $3 \times 1 = 3$
- $l = 4, r = 5, \frac{n}{i}$ 值 2, 对答案贡献 $2 \times 2 = 4$

数论分块 (整除分块)

举例: $n = 10$ 的分块过程

- $l = 1, r = 1, \frac{n}{i}$ 值 10, 对答案贡献 $10 \times 1 = 10$
- $l = 2, r = 2, \frac{n}{i}$ 值 5, 对答案贡献 $5 \times 1 = 5$
- $l = 3, r = 3, \frac{n}{i}$ 值 3, 对答案贡献 $3 \times 1 = 3$
- $l = 4, r = 5, \frac{n}{i}$ 值 2, 对答案贡献 $2 \times 2 = 4$
- $l = 6, r = 10, \frac{n}{i}$ 值 1, 对答案贡献 $1 \times 5 = 5$

数论分块 (整除分块)

举例: $n = 10$ 的分块过程

- $l = 1, r = 1, \frac{n}{i}$ 值 10, 对答案贡献 $10 \times 1 = 10$
- $l = 2, r = 2, \frac{n}{i}$ 值 5, 对答案贡献 $5 \times 1 = 5$
- $l = 3, r = 3, \frac{n}{i}$ 值 3, 对答案贡献 $3 \times 1 = 3$
- $l = 4, r = 5, \frac{n}{i}$ 值 2, 对答案贡献 $2 \times 2 = 4$
- $l = 6, r = 10, \frac{n}{i}$ 值 1, 对答案贡献 $1 \times 5 = 5$

总块数 $5 = O(\sqrt{10})$ 。

数论分块练习题

- Luogu P2261 [CQOI2007] 余数求和：求 $\sum_{i=1}^n k \bmod i$ 。

数论分块练习题

- **Luogu P2261 [CQOI2007] 余数求和**：求 $\sum_{i=1}^n k \bmod i$ 。

题解： $k \bmod i = k - \lfloor k/i \rfloor \cdot i$ ，对 $\lfloor \frac{k}{i} \rfloor$ 整除分块，每一块内部的所有 $k \bmod i$ 构成一个以 $d = -\lfloor \frac{k}{i} \rfloor$ 为公差的等差数列，每一块可以 $O(1)$ 等差数列求和。

数论分块练习题

- **Luogu P2261 [CQOI2007] 余数求和**：求 $\sum_{i=1}^n k \bmod i$ 。

题解： $k \bmod i = k - \lfloor k/i \rfloor \cdot i$ ，对 $\lfloor \frac{k}{i} \rfloor$ 整除分块，每一块内部的所有 $k \bmod i$ 构成一个以 $d = -\lfloor \frac{k}{i} \rfloor$ 为公差的等差数列，每一块可以 $O(1)$ 等差数列求和。

- 求 $\sum_{i=1}^n i \cdot d(i)$ ，其中 $d(x)$ 表示 x 的约数个数。

数论分块练习题

- **Luogu P2261 [CQOI2007] 余数求和**：求 $\sum_{i=1}^n k \bmod i$ 。

题解： $k \bmod i = k - \lfloor k/i \rfloor \cdot i$ ，对 $\lfloor \frac{k}{i} \rfloor$ 整除分块，每一块内部的所有 $k \bmod i$ 构成一个以 $d = -\lfloor \frac{k}{i} \rfloor$ 为公差的等差数列，每一块可以 $O(1)$ 等差数列求和。

- 求 $\sum_{i=1}^n i \cdot d(i)$ ，其中 $d(x)$ 表示 x 的约数个数。

$$\text{题解：} \sum_{i=1}^n i \cdot d(i) = \sum_{i=1}^n i \left(\sum_{j|i} 1 \right) = \sum_{j=1}^n \sum_{i=1}^{\lfloor \frac{n}{j} \rfloor} ji = \sum_{j=1}^n j \sum_{i=1}^{\lfloor \frac{n}{j} \rfloor} i$$

数论分块练习题

- **Luogu P2261 [CQOI2007] 余数求和**：求 $\sum_{i=1}^n k \bmod i$ 。

题解： $k \bmod i = k - \lfloor k/i \rfloor \cdot i$ ，对 $\lfloor \frac{k}{i} \rfloor$ 整除分块，每一块内部的所有 $k \bmod i$ 构成一个以 $d = -\lfloor \frac{k}{i} \rfloor$ 为公差的等差数列，每一块可以 $O(1)$ 等差数列求和。

- 求 $\sum_{i=1}^n i \cdot d(i)$ ，其中 $d(x)$ 表示 x 的约数个数。

$$\text{题解：} \sum_{i=1}^n i \cdot d(i) = \sum_{i=1}^n i \left(\sum_{j|i} 1 \right) = \sum_{j=1}^n \sum_{i=1}^{\lfloor \frac{n}{j} \rfloor} ji = \sum_{j=1}^n j \sum_{i=1}^{\lfloor \frac{n}{j} \rfloor} i$$

所以可以枚举每个整除结果块 $[L, R]$ ，记 $u = \lfloor \frac{n}{L} \rfloor = \lfloor \frac{n}{R} \rfloor$ ，那么每一块的贡献就是 $\frac{(L+R)(R-L+1)}{2} \cdot \frac{u(u+1)}{2}$ 。

BSGS 算法

- 求解离散对数 $a^x \equiv b \pmod{p}$, 其中 $\gcd(a, p) = 1$

BSGS 算法

- 求解离散对数 $a^x \equiv b \pmod{p}$, 其中 $\gcd(a, p) = 1$
- 考虑欧拉定理: $a^{\varphi(p)} \equiv 1 \pmod{p}$, ($\gcd(a, p) = 1$), 所以如果存在解 x , 那么 $x \bmod \varphi(p)$ 也是一个解, 并且在 $[0, \varphi(p))$ 的范围内不存在两个不同的解

BSGS 算法

- 求解离散对数 $a^x \equiv b \pmod{p}$, 其中 $\gcd(a, p) = 1$
- 考虑欧拉定理: $a^{\varphi(p)} \equiv 1 \pmod{p}$, ($\gcd(a, p) = 1$), 所以如果存在解 x , 那么 $x \bmod \varphi(p)$ 也是一个解, 并且在 $[0, \varphi(p))$ 的范围内不存在两个不同的解
- 因为 $O(\varphi(p)) = O(p)$, 所以方便起见, 不妨令 $m = \lceil \sqrt{p} \rceil$, 设 $x = i \cdot m - j$, 则 $a^{i \cdot m} \equiv b \cdot a^j \pmod{p}$

BSGS 算法

- 求解离散对数 $a^x \equiv b \pmod{p}$, 其中 $\gcd(a, p) = 1$
- 考虑欧拉定理: $a^{\varphi(p)} \equiv 1 \pmod{p}$, ($\gcd(a, p) = 1$), 所以如果存在解 x , 那么 $x \bmod \varphi(p)$ 也是一个解, 并且在 $[0, \varphi(p))$ 的范围内不存在两个不同的解
- 因为 $O(\varphi(p)) = O(p)$, 所以方便起见, 不妨令 $m = \lceil \sqrt{p} \rceil$, 设 $x = i \cdot m - j$, 则 $a^{i \cdot m} \equiv b \cdot a^j \pmod{p}$
- 预处理 $b \cdot a^j$ 存入哈希表 (小步), 枚举 i 查找 (大步)

BSGS 算法

- 求解离散对数 $a^x \equiv b \pmod{p}$, 其中 $\gcd(a, p) = 1$
- 考虑欧拉定理: $a^{\varphi(p)} \equiv 1 \pmod{p}$, ($\gcd(a, p) = 1$), 所以如果存在解 x , 那么 $x \bmod \varphi(p)$ 也是一个解, 并且在 $[0, \varphi(p))$ 的范围内不存在两个不同的解
- 因为 $O(\varphi(p)) = O(p)$, 所以方便起见, 不妨令 $m = \lceil \sqrt{p} \rceil$, 设 $x = i \cdot m - j$, 则 $a^{i \cdot m} \equiv b \cdot a^j \pmod{p}$
- 预处理 $b \cdot a^j$ 存入哈希表 (小步), 枚举 i 查找 (大步)
- 复杂度 $O(\sqrt{p})$

exBSGS: 解决 $\gcd(a, p) \neq 1$ 的离散对数问题

核心思想： 不断提取 a 与 p 的公因数，直到 $\gcd(a, p) = 1$ ，然后使用普通 BSGS。

exBSGS: 解决 $\gcd(a, p) \neq 1$ 的离散对数问题

核心思想: 不断提取 a 与 p 的公因数, 直到 $\gcd(a, p) = 1$, 然后使用普通 BSGS。

推导: 设 $d = \gcd(a, p)$ 。若 $d \nmid b$ 且 $b \neq 1$, 则无解 ($b = 1$ 时 $x = 0$ 是解)。

exBSGS: 解决 $\gcd(a, p) \neq 1$ 的离散对数问题

核心思想: 不断提取 a 与 p 的公因数, 直到 $\gcd(a, p) = 1$, 然后使用普通 BSGS。

推导: 设 $d = \gcd(a, p)$ 。若 $d \nmid b$ 且 $b \neq 1$, 则无解 ($b = 1$ 时 $x = 0$ 是解)。否则方程两边同除以 d :

$$a^{x-1} \cdot \frac{a}{d} \equiv \frac{b}{d} \pmod{\frac{p}{d}}$$

记 $a' = a/d$, $p' = p/d$, $b' = b/d$, 得到

$$a^{x-1} \cdot a' \equiv b' \pmod{p'}$$

令 $x' = x - 1$, 则 $a^{x'} \equiv b' \cdot (a')^{-1} \pmod{p'}$ 。

exBSGS: 解决 $\gcd(a, p) \neq 1$ 的离散对数问题

记 $a' = a/d$, $p' = p/d$, $b' = b/d$, 得到

$$a^{x-1} \cdot a' \equiv b' \pmod{p'}$$

令 $x' = x - 1$, 则 $a^{x'} \equiv b' \cdot (a')^{-1} \pmod{p'}$ 。

重复上述过程直到 $\gcd(a, p_k) = 1$, 设总共提取了 k 次, 此时方程为

$$a^{x-k} \equiv b_k \pmod{p_k}, \quad \gcd(a, p_k) = 1$$

最后用 BSGS 求解 $x - k$, 加上 k 即得原解。

exBSGS: 解决 $\gcd(a, p) \neq 1$ 的离散对数问题

记 $a' = a/d$, $p' = p/d$, $b' = b/d$, 得到

$$a^{x-1} \cdot a' \equiv b' \pmod{p'}$$

令 $x' = x - 1$, 则 $a^{x'} \equiv b' \cdot (a')^{-1} \pmod{p'}$ 。

重复上述过程直到 $\gcd(a, p_k) = 1$, 设总共提取了 k 次, 此时方程为

$$a^{x-k} \equiv b_k \pmod{p_k}, \quad \gcd(a, p_k) = 1$$

最后用 BSGS 求解 $x - k$, 加上 k 即得原解。

题目链接: **Luogu P4195 【模板】扩展 BSGS / exBSGS**

定期重构简介

- 将操作分批，每 B 个操作为一块

定期重构简介

- 将操作分批，每 B 个操作为一块
- 将待处理修改暂存，查询时暴力扫描待处理队列

定期重构简介

- 将操作分批，每 B 个操作为一块
- 将待处理修改暂存，查询时暴力扫描待处理队列
- 当待处理队列大小达到 B 时，将其整合进主数据结构并重构

定期重构简介

- 将操作分批，每 B 个操作为一块
- 将待处理修改暂存，查询时暴力扫描待处理队列
- 当待处理队列大小达到 B 时，将其整合进主数据结构并重构
- 取 $B = \sqrt{n}$ ，复杂度 $O(q\sqrt{n})$

定期重构的典型场景

- 序列中的单点插入和查询（如 LOJ#6282）

定期重构的典型场景

- 序列中的单点插入和查询（如 LOJ#6282）
- 树上动态问题（如 CF342E）

定期重构的典型场景

- 序列中的单点插入和查询（如 LOJ#6282）
- 树上动态问题（如 CF342E）
- 动态 AC 自动机（插入/删除字符串）

定期重构的典型场景

- 序列中的单点插入和查询（如 LOJ#6282）
- 树上动态问题（如 CF342E）
- 动态 AC 自动机（插入/删除字符串）
-

定期重构练习题

- LOJ#6282 数列分块入门 6：单点插入 + 单点查询。

定期重构练习题

- **LOJ#6282 数列分块入门 6**: 单点插入 + 单点查询。

题解: 用 `vector` 维护每个块, 插入时找到所在块并插入, 若块大小超过 $2\sqrt{n}$ 则拆成两块。(是不是开始讲过类似的题, 检测一下听课效果)

定期重构练习题

- **LOJ#6282 数列分块入门 6**: 单点插入 + 单点查询。

题解: 用 vector 维护每个块, 插入时找到所在块并插入, 若块大小超过 $2\sqrt{n}$ 则拆成两块。(是不是开始讲过类似的题, 检测一下听课效果)

- **CF342E Xenia and Tree**: 树上初始一个红点, 支持将某点染红, 查询某点到最近红点的距离。

定期重构练习题

- **LOJ#6282 数列分块入门 6**: 单点插入 + 单点查询。

题解: 用 vector 维护每个块, 插入时找到所在块并插入, 若块大小超过 $2\sqrt{n}$ 则拆成两块。(是不是开始讲过类似的题, 检测一下听课效果)

- **CF342E Xenia and Tree**: 树上初始一个红点, 支持将某点染红, 查询某点到最近红点的距离。

题解: 操作分块, 每 $\sqrt{m}$ 次操作后进行一次 $O(n)$ 的 BFS, 得到每个点 u 到最近红点的距离 dis_u , 查询时首先初始化答案 $ans = dis_x$, 然后遍历待处理队列, 并更新答案。

定期重构练习题

- **LOJ#6282 数列分块入门 6**: 单点插入 + 单点查询。

题解: 用 vector 维护每个块, 插入时找到所在块并插入, 若块大小超过 $2\sqrt{n}$ 则拆成两块。(是不是开始讲过类似的题, 检测一下听课效果)

- **CF342E Xenia and Tree**: 树上初始一个红点, 支持将某点染红, 查询某点到最近红点的距离。

题解: 操作分块, 每 $\sqrt{m}$ 次操作后进行一次 $O(n)$ 的 BFS, 得到每个点 u 到最近红点的距离 dis_u , 查询时首先初始化答案 $ans = dis_x$, 然后遍历待处理队列, 并更新答案。需要用欧拉路径序 $O(1)$ 求 LCA 的深度来算两点间距离。

定期重构练习题

- **CF710F String Set Queries:** 支持插入/删除模式串，查询给定文本串中模式串的总出现次数。（是不是很眼熟，前面串长分类讲过）

定期重构练习题

- **CF710F String Set Queries:** 支持插入/删除模式串，查询给定文本串中模式串的总出现次数。（是不是很眼熟，前面串长分类讲过）
题解：定期重构的思路可以直接每 $\sqrt{n}$ 次操作后重构一遍 AC 自动机，建 AC 自动机的总复杂度是 $O(L\sqrt{n})$ 。

定期重构练习题

- **CF710F String Set Queries:** 支持插入/删除模式串，查询给定文本串中模式串的总出现次数。（是不是很眼熟，前面串长分类讲过）

题解：定期重构的思路可以直接每 $\sqrt{n}$ 次操作后重构一遍 AC 自动机，建 AC 自动机的总复杂度是 $O(L\sqrt{n})$ 。

询问首先跑一遍 AC 自动机，然后对单独的串（ $\leq \sqrt{n}$ 个）每个都一遍跑 KMP，询问的总复杂度也是 $O(L\sqrt{n})$ 。

动态凸包

- 问题描述：维护点集 S ，支持给 S 加点，以及给定 k ，求 $\min\{y - kx \mid (x, y) \in S\}$ 。

动态凸包

- 问题描述：维护点集 S ，支持给 S 加点，以及给定 k ，求 $\min\{y - kx \mid (x, y) \in S\}$ 。
- 题解：如果所有询问都在加点操作之后，我们就只需要先求出下凸壳，询问在凸壳上二分就行，即用斜率为 k 的直线与下凸壳相切，得到 $y = kx + b$ 的直线，那么 $ans = b$ 。

动态凸包

- **问题描述：**维护点集 S ，支持给 S 加点，以及给定 k ，求 $\min\{y - kx \mid (x, y) \in S\}$ 。
- **题解：**如果所有询问都在加点操作之后，我们就只需要先求出下凸壳，询问在凸壳上二分就行，即用斜率为 k 的直线与下凸壳相切，得到 $y = kx + b$ 的直线，那么 $ans = b$ 。
- 但是现在有加点，所以我们可以建一个新点队列，每次询问从已有凸包和新点队列中算答案并取其中的最小值，每当新点队列大小超过 $\sqrt{N}$ 时再重构一次凸包即可。

动态凸包

- **问题描述：**维护点集 S ，支持给 S 加点，以及给定 k ，求 $\min\{y - kx \mid (x, y) \in S\}$ 。
- **题解：**如果所有询问都在加点操作之后，我们就只需要先求出下凸壳，询问在凸壳上二分就行，即用斜率为 k 的直线与下凸壳相切，得到 $y = kx + b$ 的直线，那么 $ans = b$ 。
- 但是现在有加点，所以我们可以建一个新点队列，每次询问从已有凸包和新点队列中算答案并取其中的最小值，每当新点队列大小超过 $\sqrt{N}$ 时再重构一次凸包即可。
- 给所有点预排序，重构用桶排，复杂度是 $O(N\sqrt{N})$ 的。

动态凸包

- **问题描述：**维护点集 S ，支持给 S 加点，以及给定 k ，求 $\min\{y - kx \mid (x, y) \in S\}$ 。
- **题解：**如果所有询问都在加点操作之后，我们就只需要先求出下凸壳，询问在凸壳上二分就行，即用斜率为 k 的直线与下凸壳相切，得到 $y = kx + b$ 的直线，那么 $ans = b$ 。
- 但是现在有加点，所以我们可以建一个新点队列，每次询问从已有凸包和新点队列中算答案并取其中的最小值，每当新点队列大小超过 $\sqrt{N}$ 时再重构一次凸包即可。
- 给所有点预排序，重构用桶排，复杂度是 $O(N\sqrt{N})$ 的。
- 题外话：加点凸包有基于平衡树的 $O(\log)$ 做法，删点的也有 $O(\log^2)$ 的做法，有兴趣可以自行查阅学习。

CF1491H "Yuezheng Ling and Dynamic Tree"

- 给一棵树，每个节点 i 指向父节点 a_i ，且保证 $a_i < i$ ，支持：
- 区间修改：将 $[l, r]$ 内所有节点的 a_i 减去 x ，并与 1 取 $\max$ 。
- 查询 **LCA**：询问两个节点的最近公共祖先。
- $1 \leq n, q \leq 10^5, TL = 1.5s$

CF1491H "Yuezheng Ling and Dynamic Tree"

- 题解： 考虑把树点按编号分块：

CF1491H "Yuezheng Ling and Dynamic Tree"

- 题解： 考虑把树点按编号分块：
- 给每个节点 i 维护 b_i ，表示从 i 开始跳到第一个不在同一块的祖先的编号。

CF1491H "Yuezheng Ling and Dynamic Tree"

- 题解： 考虑把树点按编号分块：
- 给每个节点 i 维护 b_i ，表示从 i 开始跳到第一个不在同一块的祖先的编号。
- 注意到当一个块 x 被减去 $cnt_x \geq \sqrt{n}$ 次后，这个块所有节点的父亲的编号至少比它本身小 $\sqrt{n}$ ，即一次就跨出去了，所以此时我们只需记录 tag_x 表示 x 这个块被减的总数，不需要维护每一项的 b_i 值了，因为此时有 $b_i = a_i - tag_x$ 。

CF1491H "Yuezheng Ling and Dynamic Tree"

- 题解： 考虑把树点按编号分块：
- 给每个节点 i 维护 b_i ，表示从 i 开始跳到第一个不在同一块的祖先的编号。
- 注意到当一个块 x 被减去 $cnt_x \geq \sqrt{n}$ 次后，这个块所有节点的父亲的编号至少比它本身小 $\sqrt{n}$ ，即一次就跨出去了，所以此时我们只需记录 tag_x 表示 x 这个块被减的总数，不需要维护每一项的 b_i 值了，因为此时有 $b_i = a_i - tag_x$ 。
- 对于 $cnt_x \geq \sqrt{n}$ 的整块修改，打懒标记 tag_x ；否则直接暴力修改并重构 b_i 。

CF1491H "Yuezheng Ling and Dynamic Tree"

- 查询 LCA 时，若两个节点不在同一块，则让更深的一方向块外跳跃；若在同一块但 b_i 不同，则一起向块外大跳；否则沿 a_i 小跳。

CF1491H "Yuezheng Ling and Dynamic Tree"

- 查询 LCA 时，若两个节点不在同一块，则让更深的一方向块外跳跃；若在同一块但 b_i 不同，则一起向块外大跳；否则沿 a_i 小跳。
- 考虑复杂度，修改操作中每个 b_i 的更新次数是 $O(\sqrt{n})$ ，每次访问的块数是 $O(\sqrt{n})$ ；查询中大跳次数和小跳次数都是 $O(\sqrt{n})$ ，总复杂度是 $O((n + q)\sqrt{n})$ 。

CF1491H "Yuezheng Ling and Dynamic Tree"

- 查询 LCA 时，若两个节点不在同一块，则让更深的一方向块外跳跃；若在同一块但 b_i 不同，则一起向块外大跳；否则沿 a_i 小跳。
- 考虑复杂度，修改操作中每个 b_i 的更新次数是 $O(\sqrt{n})$ ，每次访问的块数是 $O(\sqrt{n})$ ；查询中大跳次数和小跳次数都是 $O(\sqrt{n})$ ，总复杂度是 $O((n+q)\sqrt{n})$ 。
- 时间复杂度 $O((n+q)\sqrt{n})$ ，空间 $O(n)$ 。

总结

- 根号算法的核心是“平衡”

总结

- 根号算法的核心是“平衡”
- 分块：大段维护，小段朴素

总结

- 根号算法的核心是“平衡”
- 分块：大段维护，小段朴素
- 莫队：离线排序，双指针移动

总结

- 根号算法的核心是“平衡”
- 分块：大段维护，小段朴素
- 莫队：离线排序，双指针移动
- 根号分治：大小分类，分别处理

总结

- 根号算法的核心是“平衡”
- 分块：大段维护，小段朴素
- 莫队：离线排序，双指针移动
- 根号分治：大小分类，分别处理
- 整除分块：连续相同值批量计算

总结

- 根号算法的核心是“平衡”
- 分块：大段维护，小段朴素
- 莫队：离线排序，双指针移动
- 根号分治：大小分类，分别处理
- 整除分块：连续相同值批量计算
- 定期重构：批量操作，定期整合

谢谢！